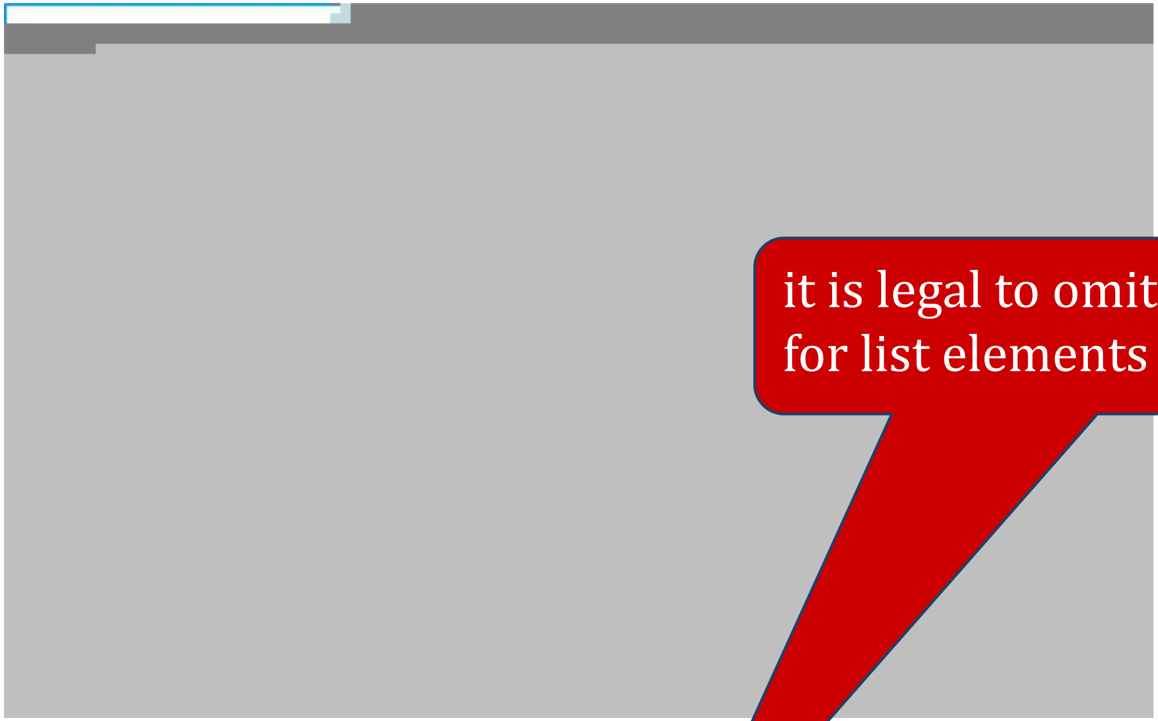
Organizing A Page's Content

Presented by
Nguyễn Đức Thái
Ho Chi Minh City University of Technology

Outline

- Introduction
- Unordered Lists
- Descendant Selectors
- Ordered Lists
- Figures
- Organizational Elements
- Section, Article, and aside Elements
- Header and Footer Elements
- Child Selectors
- CSS Inheritance

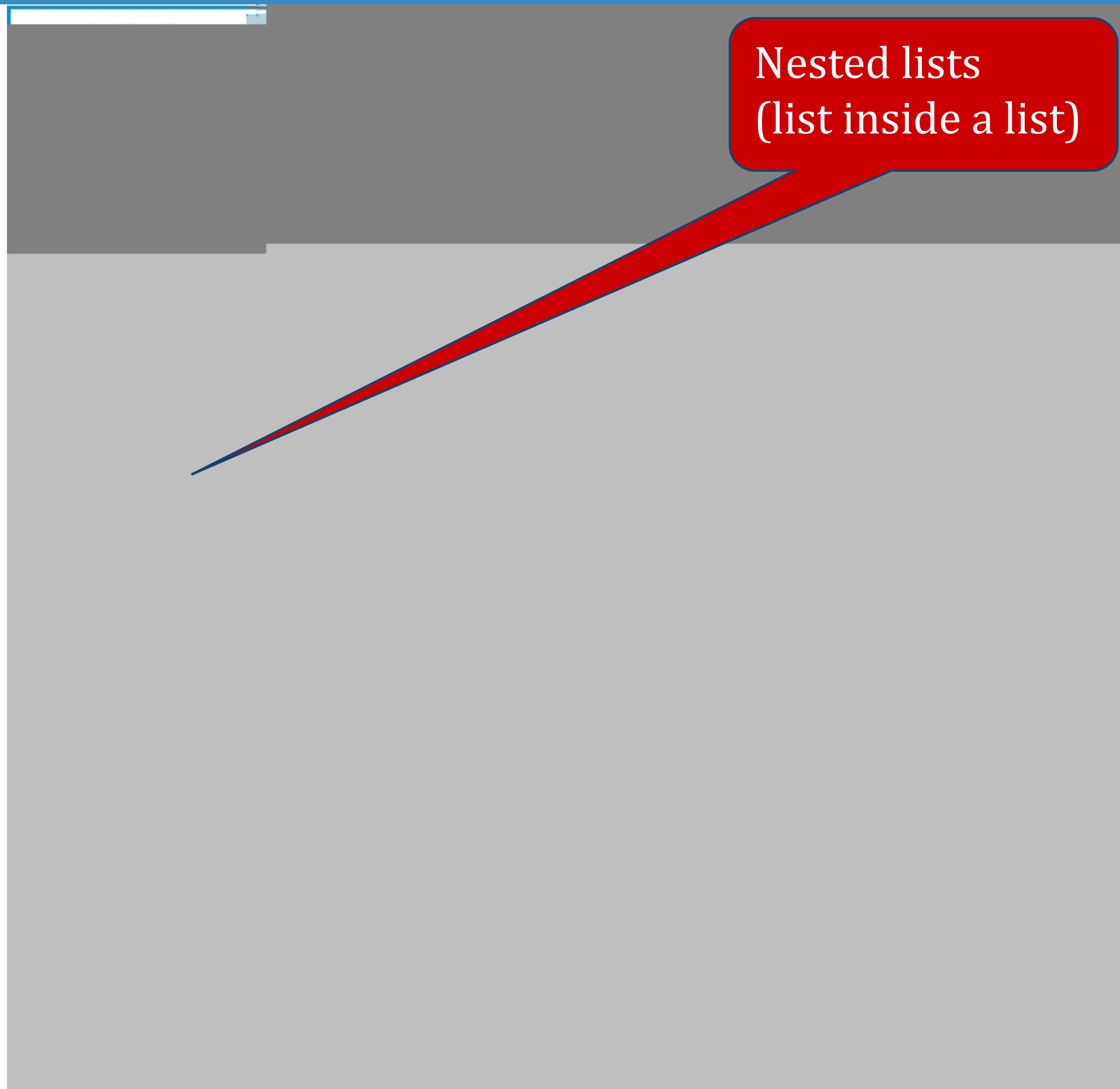
Unordered Lists



it is legal to omit the end tag for list elements

```
<ul>
    <li>Wake up at 9ish.</li>
    <li>Go to school.</li>
</ul>
```

Parent and Child Elements



Nested lists
(list inside a list)

Symbol for Unordered List Items

- *According to the W3C*, the *default symbol* for unordered list items is a ***bullet*** for all levels in a nested list, but the major browsers typically use **bullet**, **circle**, and **square** symbols for the *different levels* in a nested list.
- Because the official symbol defaults and the browser symbol defaults are different, you should **avoid** *relying on them*.
- *Instead*, you should use **CSS**'s list-style-type property to *explicitly* specify the symbols used in your web page lists.

Symbol for Unordered List Items

- For unordered lists, the *most popular values* for the *list-style-type* property are **none**, **disc**, **circle**, and **square**.
- none ➔ browser displays *no symbol* next to each list item.
- disc ➔ generates *bullet symbols*.
- circle & square ➔ generate *hollow circles* and filled-in *squares*, respectively

```
<style>
    ul {list-style-type: circle;}
    ul ul {list-style-type: square;}
</style>
```

Descendant Selectors

- For each pair of adjacent selectors, the browser searches for a pair of elements that match the selectors such that the second element is contained within the first element's start tag and end tag.
- When an element is *inside* another element's start tag and end tag, we say that the element is a descendant of the outer element.

```
<style>
    ul {list-style-type: disc;}
    ul ul {list-style-type: square;}
    ul ul ul {list-style-type: none;}
</style>
```

Descendant Selectors

- If you ever have **two** or **more** CSS rules that *conflict*, the **more specific** rule **wins**.

- That's a general principle of programming and you should remember it—the more specific rule wins.

Ordered Lists

- How to generate list items that have *numbers* and *letters* next to them.
- Numbers and letters indicate that the **order** of the list items is important, and that changing the order would change the list's meaning.
- Those types of lists are referred to as *ordered lists*.

```
<ol>
	<li>Personal experiences</li>
	<li>What friends say</li>
</ol>
```

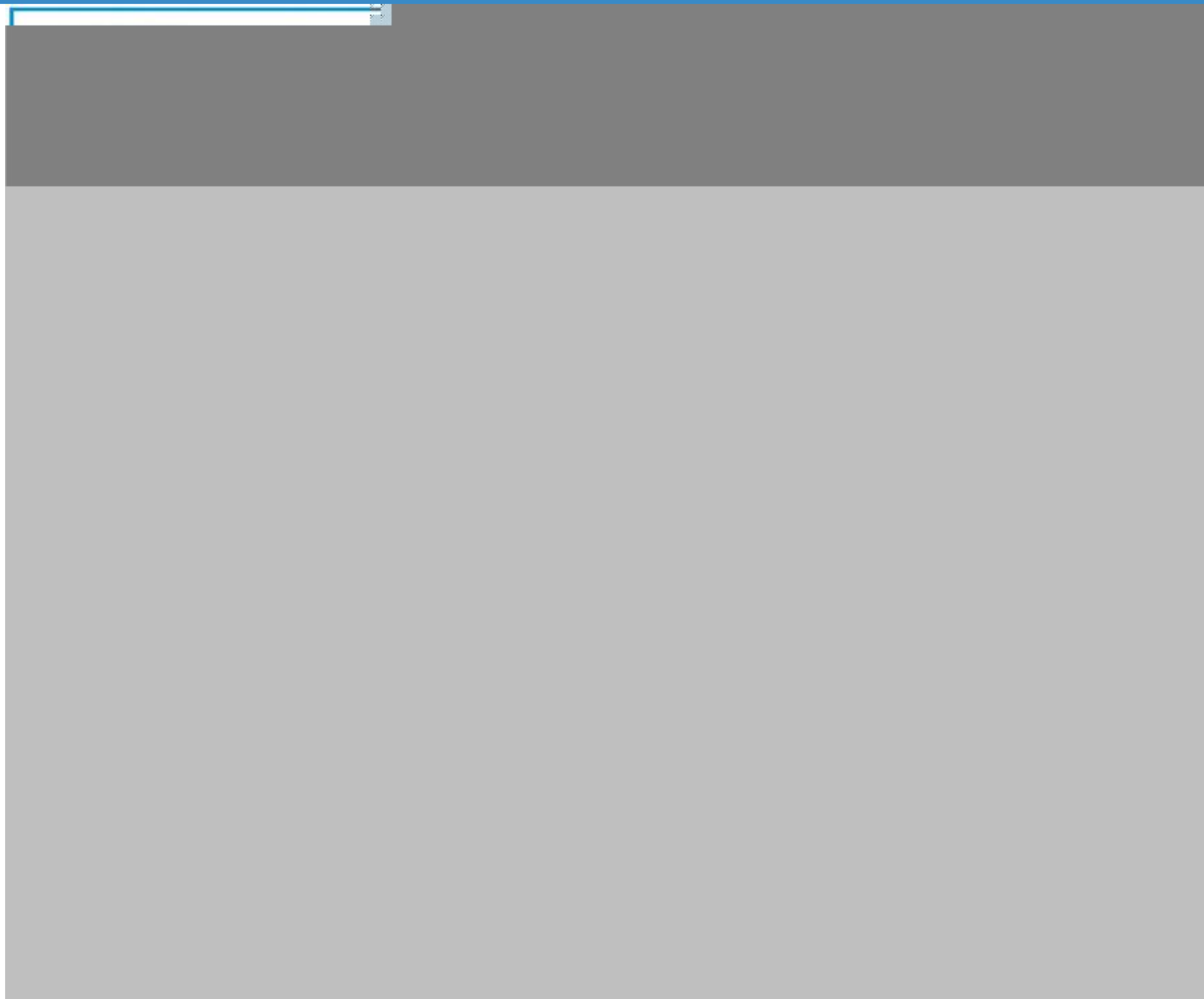
```
<style>
```

Figures

- Typically, a **figure** holds *text*, *programming code*, an *illustration*, a *picture*, or a *data table*. As with all figures, the figure element's content should be self-contained, and it should be referenced from elsewhere in the web page.

```
figure {
     border: solid crimson;
     padding: 6px;
}
```

Figures

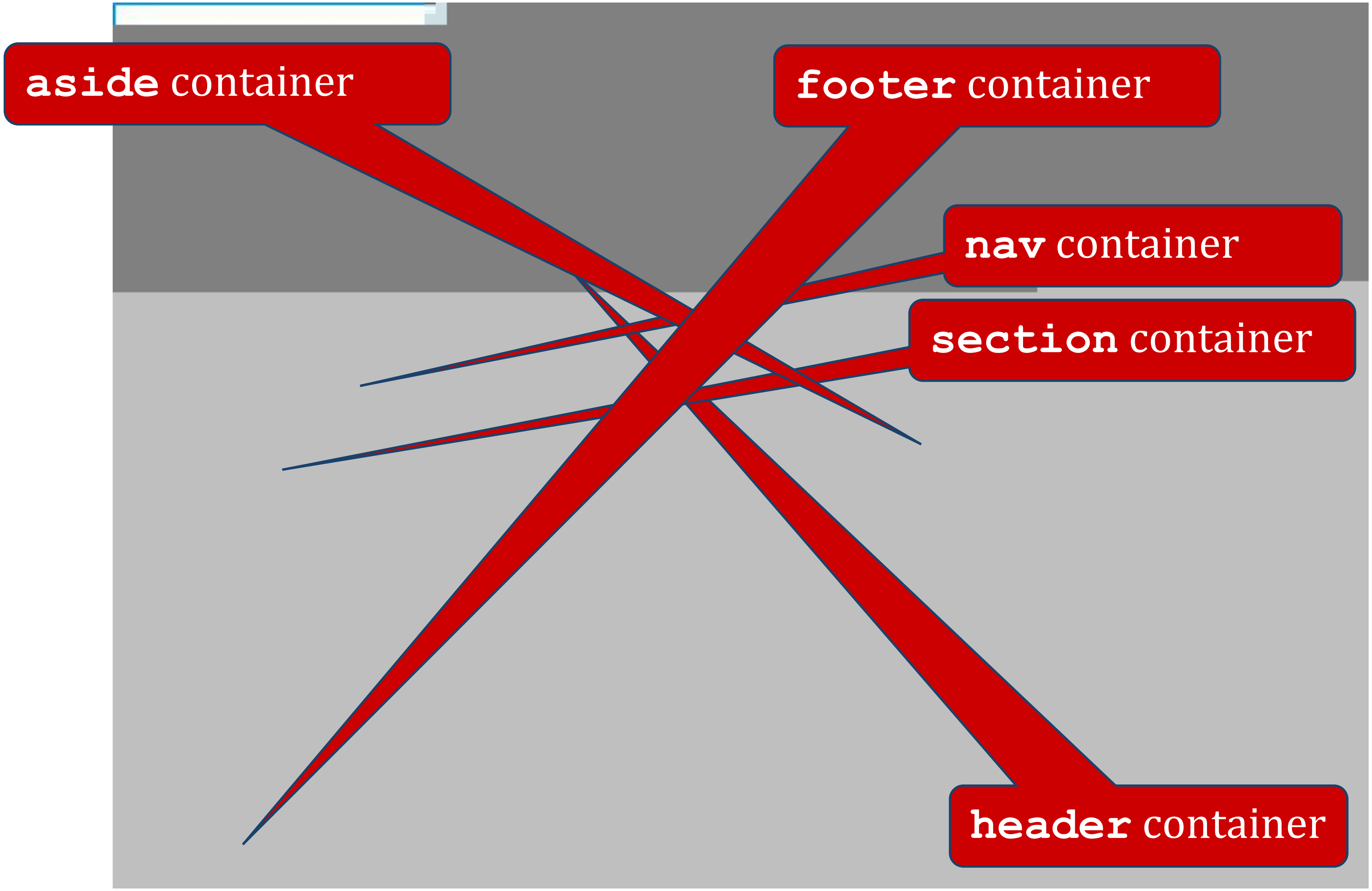


Figures

Organizational Elements

- To group web page content into sections so that we can use CSS and JavaScript to manipulate their content more effectively.
- The organizational elements:
  - `section`
  - `article`
  - `aside`
  - `nav`
  - `header`
  - `footer`

Organizational Elements



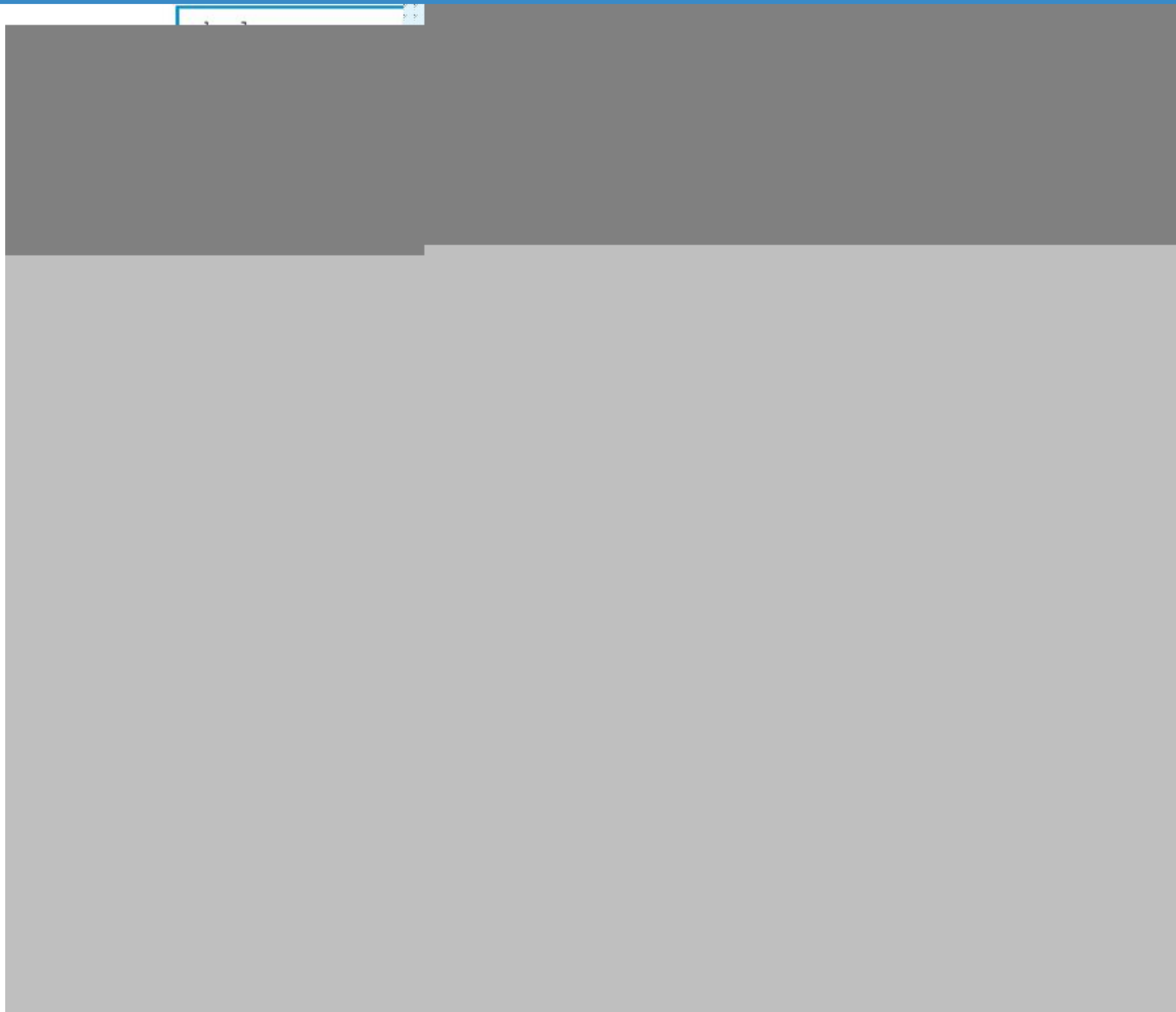
Organizational Elements

`section` element

- **`section`**: ➔ container: used to group together a section of a web page.

- *Typically*, a section will contain a *heading* and *one or more paragraphs*, with the *heading* saying *something about the common theme*

section element

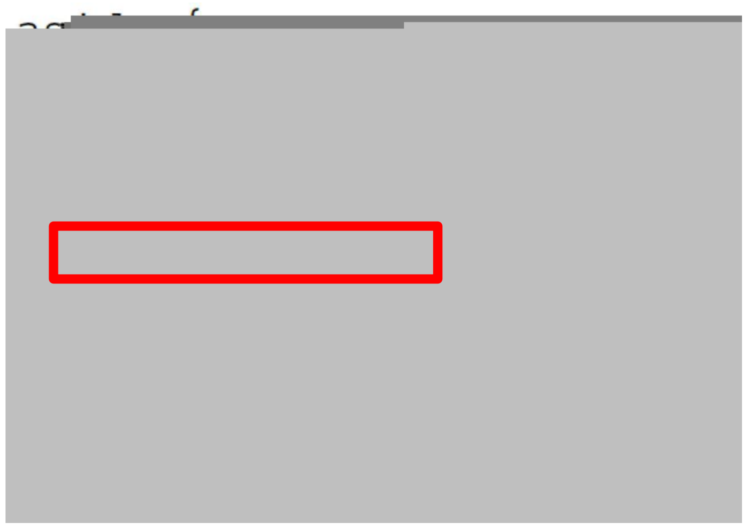


`article` element

- **`article`** → grouping together *one* or *more sections* such that the *group of sections* form an *independent part* of a web page.

`aside` element

- **`aside`** → to group together *content* that has *something to do* with the rest of the web page, but it isn't part of the main flow.
- Typically, you should position an **`aside`** element at the *right* or *left*.



nav & **a** element

- **nav** ➔ a container that normally contains *links* to other web pages or to other parts of the current web page
- **nav** ➔ navigation
- **a** ➔ implements a link

`header` & `footer` element

- **`header`** → for grouping together *one* or *more* *heading elements* (h1-h6)
- Normally, the **`header`** is associated with a *section*, an *article*, or the *entire web page*.
- To form that association, the **`header`** element must be positioned **within** its associated content container.
- Typically, that means *at the top of the container*, but it is legal and sometimes appropriate to have it positioned lower.

header & footer element

`header` & `footer` element

- **`footer`** ➔ for grouping together information to form a footer.
- *Typically*, the **`footer`** holds content such as *copyright data*, *author information*, or *related links*.
- The **`footer`** should be *associated* with a **`section`**, an **`article`**, or the entire web page.
- To form that association, the **`footer`** element must be *positioned* within its *associated content container*.
- *Typically*, that means at the bottom *of the container*, but it is legal and sometimes appropriate to have it positioned elsewhere.

header & footer element

Child Selectors

- A *child selector* is a more refined version of a descendant selector.

list-

sect i

`combinator`, used to combine the elements

CSS Inheritance

- CSS inheritance is *when* a CSS property value *flows down* from a **parent** element to child (children)
- It parallels the inheritance of genetic characteristics (e.g., height and eye color) from a biological parent to a child.
- Some *CSS properties* are *inheritable* and some are *not*.
- CSS properties **inheritable**:

References

- W3 Schools – w3schools.com
- Web Programming with HTML5, CSS, and JavaScript – John Dean – Jones & Bartlett Publishers, 2019.